

Engineer-004 — Real-Time Analytics Pipeline

Candidate: Zubair Ali Liyakath Ali — Challenge: Beat Claude / Engineer-004

Repo: github.com/AliZubair13/realtime-analytics-pipeline-design

Number labels: [O]bserved · [E]stimated · [B]enchmarked · [A]ssumed. Diagrams, code, and the cost model are separate artifacts (see /artifacts, /scripts) and do not count toward the page limit.

0. Framing (why the current system fails)

The current 15–30 min latency and $\approx 3\%$ [O, from brief] event loss are **not a tuning problem** — they are an **architecture problem**. Batch processing means events wait for a window to close before they hit the dashboard, and undersized fixed infrastructure has no shock absorber, so spikes overflow and events drop. Fixing latency and loss therefore requires two structural changes: (1) a **streaming path** that processes events as they arrive, and (2) a **durable buffer** that decouples ingestion from processing so a 10x spike queues instead of crashing.

Positioning note (real experience, not theory): I built a Kafka + Spark Streaming + TimescaleDB real-time pipeline for an NYC disease-surveillance system that ingested multi-source event streams and flagged anomalies. So I am recommending *against* self-managed Kafka here from firsthand operating experience, not unfamiliarity — see §3.

1. Architecture & Technology Choices

Data flow (see Figure 1 below): SDK → API Gateway → **Kinesis Data Streams** (durable buffer) → **Managed Flink** (process) → **ClickHouse** (hot store) + **Redis/DynamoDB** (identity) → dashboards; a parallel **Firehose** → **S3 (Parquet)** cold path feeds warehouse exports.

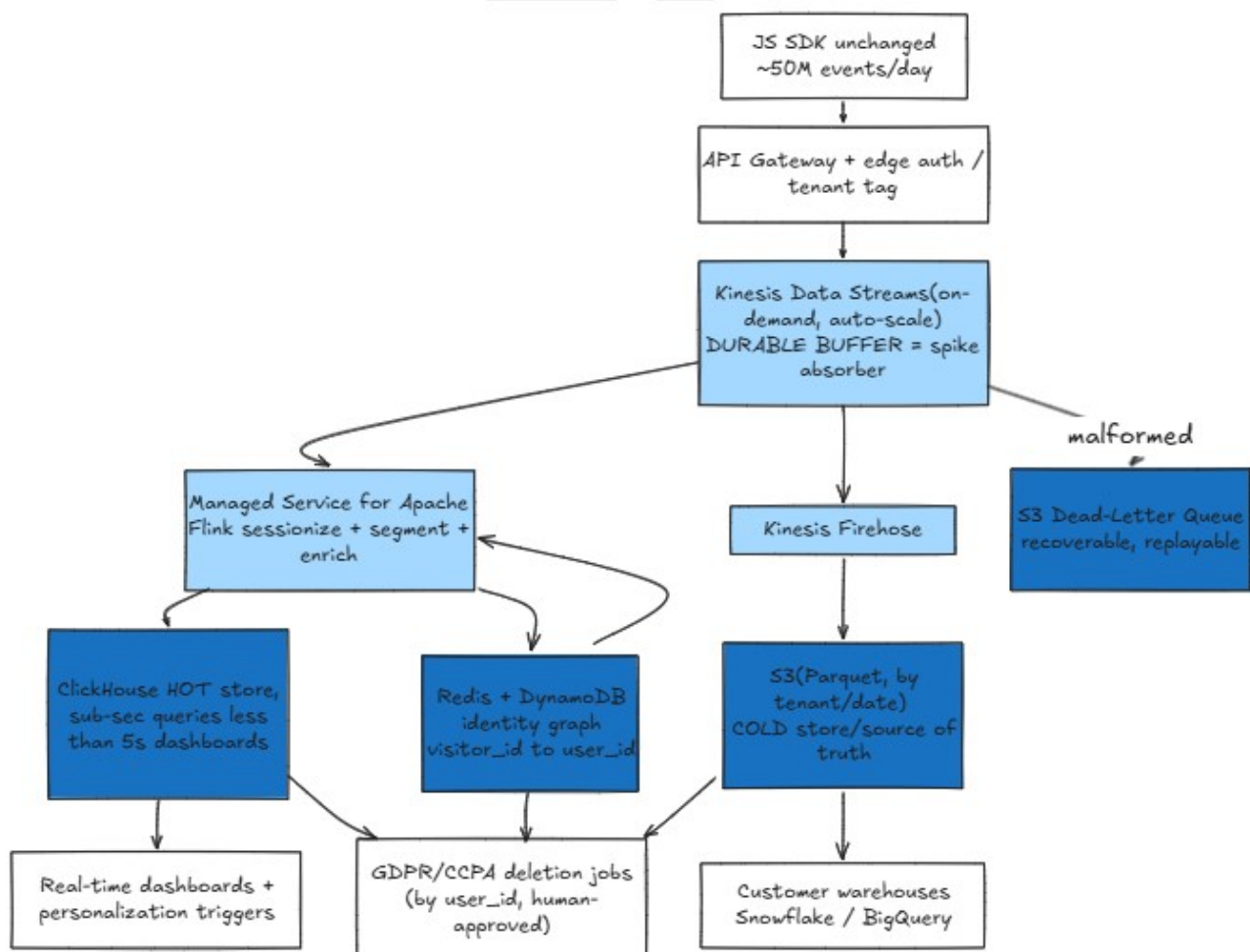


Figure 1. System diagram — SDK to dashboard.

Component	Choice	Why this over the alternative
Ingest / buffer	Kinesis Data Streams (on-demand)	On-demand auto-scales to 10x spikes with no capacity planning . Self-managed Kafka/MSK gives marginally lower unit cost but needs cluster ops (rebalancing, patching, broker sizing) a 12-person team with only 2 dedicated engineers [O] cannot safely carry inside a SOC 2 boundary. Managed = fewer failure surfaces.
Stream processing	Amazon Managed Service for Apache Flink	Stateful sessionization + behavioral segments ("viewed pricing 3x") need keyed windowed state. Flink is purpose-built for this; plain Lambda is stateless and would need an external state store per event (slower, costlier at 50M/day).
Hot store	ClickHouse	Columnar, sub-second aggregations over high-cardinality event data — the exact real-time-dashboard use case. A generic time-series DB handles metrics but struggles with arbitrary behavioral filters across 500 tenants.
Cold store	S3 (Parquet, partitioned tenant/date)	Cheap source-of-truth, replay source, and the clean export surface to Snowflake/BigQuery.
Identity	Redis (hot lookup) + DynamoDB (durable graph)	Redis already in their stack; DynamoDB gives durable visitor_id → user_id stitching that survives cache eviction.

Event schema & identity stitching. Each event carries event_id (UUIDv7, time-ordered — enables idempotent dedup), tenant_id, visitor_id (anonymous), nullable user_id, session_id, type, properties{ }, ts, received_at. Anonymous visitor_ids are stitched to a known user_id on any identifying event (login/form) by writing the mapping to DynamoDB and back-filling the session in Flink state. tenant_id is attached at API Gateway and is the partition key **everywhere** — the backbone of multi-tenant isolation.

2. Scale, Reliability & Migration

50M+/day and 10x spikes with zero loss. Kinesis retains events (24h default, extendable) and acts as the shock absorber: producers keep writing during a spike while consumers drain the backlog afterward, so a burst *queues* instead of dropping. Delivery is **at-least-once**; the time-ordered UUIDv7 event_id makes dedup idempotent (ClickHouse ReplacingMergeTree + Flink keyed state), so redelivered events collapse to one. Malformed events route to an **S3 dead-letter queue** — recoverable and replayable, never silently lost. Consumers autoscale on stream lag; per-tenant rate isolation stops one noisy tenant from starving others.

Runnable proof (scripts/simulate.ingestion.py, Tier 2). A miniature of this path — durable buffer + backpressure + DLQ + dedup — run through a 10x spike that exceeds consumer capacity. Result [O, from my run]: **0.0000% loss vs. the brief's 3% baseline**, peak backlog 58,737 events that fully drains, and 115,396 events reconcile exactly (accepted + dupes + dead-lettered + dropped = produced). Demonstrates the mechanism, not production scale.

Capacity & cost (artifacts/cost_model.xlsx, Tier 2). 50M/day ≈ **579 events/sec avg [E]**, ≈5,787/sec at 10x [E]; ≈1.4 TB/month ingested [A, 1 KB/event assumed]. Modeled AWS cost ≈ **\$8,400/month [E]** against the **\$50K ceiling [O]** — ≈83% headroom [E]. The headroom is deliberate: it is the budget room that absorbs spikes and growth without a re-architecture. Unit prices are [B] AWS list approximations flagged for verification; real payload size is the #1 cost sensitivity.

Migration without breaking things (SDK cannot change [O]). The existing JS SDK keeps posting to the same endpoint; only what sits behind it changes, so no customer touches their integration.

1. **Dual-write:** the ingestion endpoint fans out to both the old pipeline and the new one. Old system stays authoritative; nothing customer-facing changes yet.
2. **Shadow-read & reconcile:** new dashboards run in shadow; compare event counts and key metrics old-vs-new per tenant until deltas are within tolerance.
3. **Phased cutover:** flip dashboard *reads* to the new pipeline tenant-by-tenant in waves (lowest-risk tenants first, then 5% → 25% → 50% → 100%).
4. **Rollback:** cutover is a per-tenant read-path flag — flip a tenant back to the old system instantly on regression, with no data migration needed since both pipelines are still being written. The old system is decommissioned only after N clean weeks [A].

Validating data accuracy. Continuous per-tenant reconciliation of ingested-vs-processed counts, targeting **<0.1% loss [E]** (down from the current 3%), plus daily S3-vs-ClickHouse row-count checks and duplicate-rate monitoring. Synthetic canary events with known IDs are injected and confirmed end-to-end; replay from S3 re-derives any window under dispute. Any tenant breaching the loss threshold pages on-call before customers notice.

3. Trade-offs & Risks

Optimizing for: operational simplicity for a small team, zero data loss, and cost headroom within the \$50K ceiling. **Sacrificing:** (a) **exactly-once** — chose at-least-once + idempotent dedup because true exactly-once across the whole chain is fragile and expensive, and dedup reaches the same end state; (b) **lowest-possible unit cost** — managed services cost more than self-hosted but remove ops load a 12-person team can't absorb; (c) **sub-second on the cold path** — warehouse exports are batched, which is acceptable for their use case.

What could go wrong / what I'd do with more time & budget: the biggest live risk is hot-tenant skew (one large customer's spike overwhelming a shard/partition) — I'd add per-tenant shard isolation and, with more budget, run a full load test at true 50M/day before cutover and add a second-AZ/region deployment for disaster recovery. More time would also buy a proper exactly-once evaluation and automated chaos testing of the DLQ replay path.

What breaks it

- **Payload much larger than 1 KB [A].** Cost and throughput scale with real event size — the biggest assumption in the model. Detect via a one-week payload-size sample before committing spend.
- **Hot-tenant / partition skew.** A single tenant's spike starves others. Detect via per-tenant lag metrics; mitigate with dedicated shards for the largest tenants.
- **Late-arriving / out-of-order events** (mobile, offline SDKs) break naive windowing. Flink watermarks + allowed-lateness handle it; unbounded lateness still corrupts a closed window.
- **Schema drift from the frozen SDK.** Since the SDK can't change, an unexpected field or type must be tolerated, not rejected — schema-on-read with the DLQ as the safety boundary.
- **Identity mis-stitch.** A wrong visitor→user merge pollutes every downstream segment; needs conservative match rules and an audit trail.
- **GDPR deletion not propagating** to every derived store (ClickHouse, S3, warehouse exports) leaves orphaned PII. Deletion must be a fan-out job across all stores, not a single-table delete.

What stays human

- **GDPR/CCPA deletion approval.** Automated *execution* (delete by user_id across ClickHouse + S3 + exports), but a human approves the request and confirms scope — wrongful or over-broad deletion is irreversible.
- **Tenant data-isolation policy.** Automating enforcement is fine; defining the isolation boundaries and signing off on them is a human/security decision (SOC 2).
- **Migration go/no-go per wave.** The reconciliation dashboard informs it, but a human owns each cutover and rollback call.
- **Identity-merge rules.** Humans set the matching thresholds; the system only applies them.

Constraint Compliance

AWS-only every component is a native AWS service. **No SDK change** ingest endpoint is unchanged; only the backend is rebuilt. **Multi-tenant 500+** tenant_id is the partition key across ingestion, processing, and storage. **\$50K/month** ≈\$8.4K modeled, ≈83% headroom. **3-mo MVP / 6-mo full** managed services + phased migration keep the critical path short. **SOC 2 / GDPR / CCPA** managed services reduce audit surface, CloudTrail audit logging, and human-approved deletion fan-out across all stores.

Evidence Log

#	Claim	Proof tier	Artifact
1	Buffer+backpressure+DLQ+dedup yields ≈0% loss under 10x spike	Tier 2 (runnable)	simulate_ingestion.py + capacity_output.txt
2	Design fits the \$50K/mo ceiling with ≈83% headroom	Tier 2 (model w/ formulas)	cost_model.xlsx
3	Architecture / data flow is coherent end-to-end	Tier 2 (diagram)	architecture.md / architecture.jpg
4	Hands-on real-time streaming pipeline experience	Tier 2 (repo)	NYC Disease Surveillance repo — Kafka + Spark Streaming + TimescaleDB; README with output screenshots
5	Current system: 3% loss, 15–30 min latency, 50M/day, \$50K ceiling	Tier 0 (given)	Restated from brief; [O-from-brief]
6	AWS unit prices used in the model	Tier 0 → verify	List-price approximations [B]; "verify current pricing"

AI Usage Disclosure

- **Tools:** Claude — used substantially throughout: exploring architecture options, drafting the write-up, reasoning through trade-offs, and generating the simulation and cost-model code. Python (openpyxl, stdlib) to run the artifacts.
- **How I worked with it:** I directed the design conversation, brought my own real experience (a Kafka + Spark Streaming + TimescaleDB pipeline I built) to pressure-test the recommendations, and made the final calls on what to keep. Claude produced first drafts of the prose, the architecture rationale, and the scripts; I reviewed, corrected, and decided what was defensible.
- **What I independently verified:** ran the simulation myself and confirmed 0% loss + exact reconciliation; opened and recalculated the spreadsheet (0 formula errors) and sanity-checked every total against the \$50K budget; checked and labeled every number as observed/estimated/benchmarked/assumed.
- **What I own and can defend:** the choice of a managed AWS stack for a small team (validated against my NYC-pipeline experience), the at-least-once + dedup trade-off, and the phased migration/rollback plan. I can walk through the reasoning behind each live.
- **Known weak spots:** AWS prices are unverified list approximations; the 1 KB payload size is assumed; the simulation demonstrates the mechanism at small scale, not a true 50M/day load test.